

Khóa học lập trình Java nâng cao

API lưu trữ dữ liệu Java



Khoa Công nghệ Thông tin
Đại học Công nghiệp Thành phố Hồ Chí Minh

Mục tiêu của buổi học

- Giới thiệu về API Lưu trữ Dữ liệu Java
- ORM - Bộ ánh xạ đối tượng/quan hệ
 - o Các thực thể
 - o EntityManager & Ngữ cảnh bền vững
 - o Đơn vị lưu trữ
 - o Ngoại lệNgôn ngữ truy vấn lưu trữ Java
- OGM - Trình ánh xạ đối tượng/lưới
 - o Giới thiệu
 - o OGM cho MongoDB





Giới thiệu

Giới thiệu

- Trước đây chúng ta đã tìm hiểu về
 - o JDBC
 - o Đối tượng truy cập dữ liệu (DAO) và Đối tượng truyền dữ liệu (DTO)
1. Trong JDBC, chúng ta đã "mã hóa cứng" SQL vào ứng dụng của mình.
 2. Sau đó sử dụng Nguồn dữ liệu/Gộp kết nối.
 3. Sau đó sử dụng DAO/DTO
 4. Nhưng điều này chỉ "che giấu" việc triển khai khỏi logic nghiệp vụ của chúng ta.
Bạn vẫn triển khai DAO bằng JDBC.

Các vấn đề chưa được giải quyết

- Tuy nhiên,

Chúng ta vẫn cần hiểu rõ nhiều chi tiết triển khai (ví dụ: kết nối, câu lệnh, tập kết quả, v.v.)

Vậy còn các mối quan hệ thì sao? Sự kết nối? Sự kế thừa?

- o Không khớp trở kháng cơ sở dữ liệu đối tượng

- J2EE đã cố gắng giải quyết vấn đề này bằng "Entity Enterprise".

JavaBeans (EJB)"

- o Bao gồm các lựa chọn thay thế đơn giản hơn

Công cụ Ánh xạ Đối tượng Quan hệ (ORM):

- o ví dụ: Đối tượng dữ liệu Java (JDO), **Hibernate**, iBatis, TopLink

Ghi chú: Lỗi khớp quan hệ đối tượng

- Không tương thích giữa đối tượng và

quan hệ: Kiểu dữ liệu SQL và kiểu dữ liệu Java khác nhau.

- Các cơ sở dữ liệu cũng hỗ trợ các kiểu dữ liệu SQL khác nhau
- Thường định nghĩa các kiểu dữ liệu nội bộ riêng, ví dụ như kiểu NUMBER của Oracle.
kiểu

- Các kiểu dữ liệu phải được ánh xạ giữa Java và SQL/Cơ sở dữ liệu •

Các kiểu dữ liệu JDBC (Generic SQL) được định nghĩa trong `java.sql.Types`

- Các kiểu dữ liệu Java rất phong phú; các kiểu dữ liệu SQL thì hạn chế

hơn o Làm thế nào để ánh xạ lớp với bảng? 1:1?

1:n? o Làm thế nào để ánh xạ các cột với các thuộc

tính của lớp? o Hỗ trợ BLOB? Truyền

dữ liệu theo luồng? o Làm thế nào để thiết kế hướng đối tượng ở đây? Còn về kế thừa thì sao?

Trừu tượng hóa? Tái sử dụng?

Giới thiệu về JPA

- Java Persistence API (JPA) là gì?
 - Công nghệ lưu trữ dữ liệu trong cơ sở dữ liệu dành cho Java
 - Công cụ ánh xạ đối tượng-quan hệ (ORM)
 - Hoạt động với các thực thể POJO
 - Tương tự như Hibernate và JDO
- JPA ánh xạ các lớp Java tới các bảng cơ sở dữ liệu
 - Ánh xạ các mối quan hệ giữa các bảng thành các liên kết giữa các lớp
- o Cung cấp chức năng CRUD
 - Tạo, đọc, cập nhật, xóa

Lịch sử của JPA

- Lịch sử của JPA o

Được tạo ra như một phần của EJB 3.0 trong JSR

220 o Phát hành vào tháng 5 năm 2006 như một

phần của Java EE 5 o Có thể được sử

dụng như một thư viện độc lập • API chuẩn với nhiều triển khai

o OpenJPA - <http://openjpa.apache.org/> o

Hibernate - <http://www.hibernate.org> o

TopLink JPA - <http://www.oracle.com/technology/jpa> o JPOX -

<http://www.jpox.org/>

Triển khai JPA

- Triển khai tham chiếu: TopLink (GlassFish) dự án)
- Hầu hết các nhà cung cấp ORM hiện nay đều có giao diện JPA.
 - o Hibernate-JPA,
 - o EclipseLink (dựa trên TopLink), o OpenJPA (dựa trên BEA Kodo)
- Hoàn toàn là mã nguồn mở (theo giấy phép CDDL)
Bất kỳ ai cũng có thể tải xuống/sử dụng mã nguồn hoặc mã nhị phân trong quá trình phát triển hoặc sản xuất.



API lưu trữ dữ liệu Java

Các thực thể

Cấu trúc giải phẫu của một thực thể

- Một **thực thể** là một đối tượng Java đơn giản (POJO)
 - Lớp đại diện cho một **bảng** trong cơ sở dữ liệu quan hệ.
 - **Các thể hiện** tương ứng với **các hàng**
 - Yêu cầu:
 - o Được chú thích bằng chú thích `javax.persistence.Entity`
 - o Hàm tạo **công khai** hoặc **được bảo vệ**, không có đối số (parameterless)
 - o Lớp không được khai báo **là final**
- Không được khai báo bất kỳ phương thức hoặc biến thể hiện nào tồn tại .

cuối cùng

Yêu cầu đối với các thực thể

- Có thể được tuần tự hóa, nhưng không bắt buộc.
 - Chỉ cần thiết nếu được truyền theo giá trị (trong một cuộc gọi từ xa)
- Các thực thể có thể kế thừa cả lớp thực thể và lớp không phải thực thể
- Các lớp không phải thực thể có thể kế thừa lớp thực thể
- Các biến thể hiện bền vững phải được khai báo là `private`, `protected`, hoặc `package-private` (khả năng hiển thị mặc định)
- Không yêu cầu giao diện nghiệp vụ/gọi lại • Ví dụ:

```
@Entity
class Person{
    . . .
}
```

Các trường và thuộc tính bền vững

- Có thể truy cập trạng thái bền vững của một thực thể:

thông qua các biến thể hiện của thực thể

thông qua các thuộc tính kiểu JavaBeans (getters/setters)

- Các loại được hỗ trợ:

các kiểu dữ liệu nguyên thủy, chuỗi String, các kiểu dữ liệu có thể tuần tự hóa khác, các kiểu liệt kê

các thực thể và/hoặc tập hợp các thực thể khác

các lớp có thể nhúng

- Tất cả các trường không được chú thích bằng `@Transient` hoặc không được đánh dấu là Java

Dữ liệu tạm thời sẽ được lưu trữ vào kho dữ liệu!

Khóa chính trong các thực thể

- Mỗi thực thể phải có một mã định danh đối tượng duy nhất (duy trì).
mã định danh)

@Entity

lớp công khai Nhân viên {

@Id private int id;

Tên chuỗi riêng tư;

Thông tin cá nhân, ngày tháng, tuổi tác;

public int getId() { return id; }

public void setId(int id) { this.id = id; }

. . .

}



Khóa chính

Nhận dạng bền vững

- Mã định danh (id) trong thực thể = khóa chính trong cơ sở dữ liệu
- Xác định duy nhất thực thể trong bộ nhớ và trong cơ sở dữ liệu
- Các kiểu định danh bền vững:

- o ID đơn giản - trường/thuộc tính đơn lẻ

```
@Id int id;
```

- o Mã định danh phức hợp - nhiều trường/thuộc tính

```
@Id int id;
```

```
@Id Tên chuỗi; o
```

- ID nhúng - trường đơn của kiểu lớp PK

```
@EmbeddedId ID khóa chính của nhân viên;
```

Tạo định danh

- Mã định danh có thể được tạo trong cơ sở dữ liệu bằng cách chỉ định `@GeneratedValue` trên định danh

- Bốn chiến lược tạo thế hệ được xác định trước:

TỰ ĐỘNG, NHẬN DẠNG, TRÌNH TỰ, BẢN

- Máy phát điện có thể đã tồn tại từ trước hoặc được tạo ra.
- Việc chỉ định chiến lược là AUTO cho biết nhà cung cấp sẽ chọn một chiến lược.
- Ví dụ

`@Nhận dạng`

```
@GeneratedValue(strategy=GenerationType.AUTO)  
private int id;
```


Tùy chỉnh đối tượng thực thể

- Trong hầu hết các trường hợp, các thiết lập mặc định là đủ.
- Theo mặc định, **tên bảng** tương ứng với **tên không đủ điều kiện** của lớp

- Tùy chỉnh:

```
@Entity
@Table(name = "FULLTIME_EMPLOYEE")
public class Employee{ .. }
```

- Có thể tùy chỉnh các giá trị mặc định của cột bằng cách sử dụng **@Column**.
chú thích

```
@Id @Column(name = "EMPLOYEE_ID", nullable = false) private
String id;

@Column(name = "FULL_NAME" nullable = true, length = 100) private
String name;
```

Mối quan hệ giữa các thực thể

- Có bốn loại bội số quan hệ:
 - o @OneToOne
 - o @OneToMany
 - o @ManyToOne
 - o @ManyToMany
- Chiều của mối quan hệ có thể là: o hai chiều
 - bên sở hữu và bên đối lập o một chiều -
chỉ bên sở hữu
- Bên sở hữu chỉ định ánh xạ vật lý

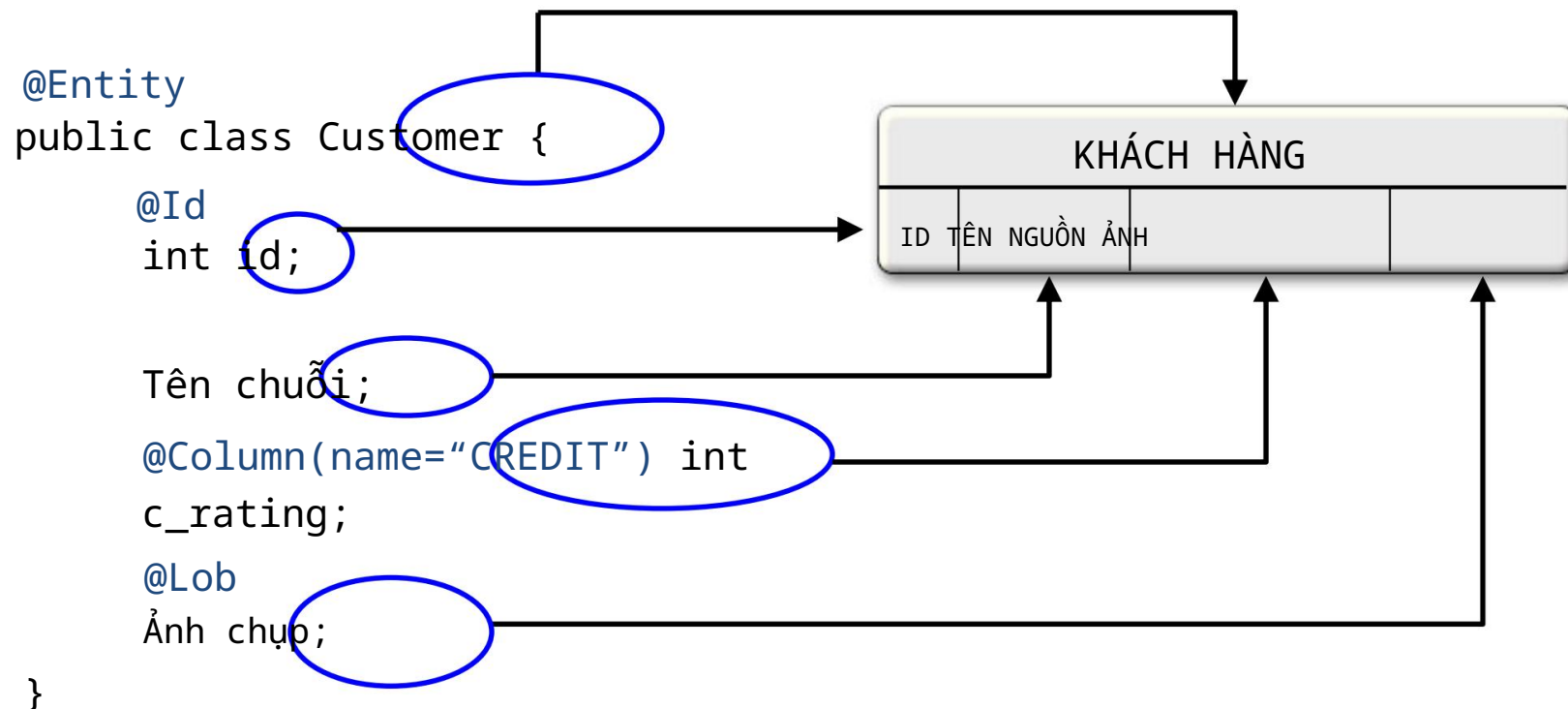
Thuộc tính quan hệ thực thể

- JPA hỗ trợ cập nhật/xóa theo kiểu thác nước
 - **CascadeType**
 - TẤT CẢ, LƯU TRỮ, HỢP NHẤT, XÓA, LÀM MỚI
- Bạn có thể khai báo chiến lược hiệu suất để sử dụng khi truy xuất các hàng liên quan.
 - **FetchType**
 - LƯỜI BIẾNG, HÁO HỨC
 - (Lười biếng có nghĩa là không tải hàng cho đến khi thuộc tính được truy xuất)

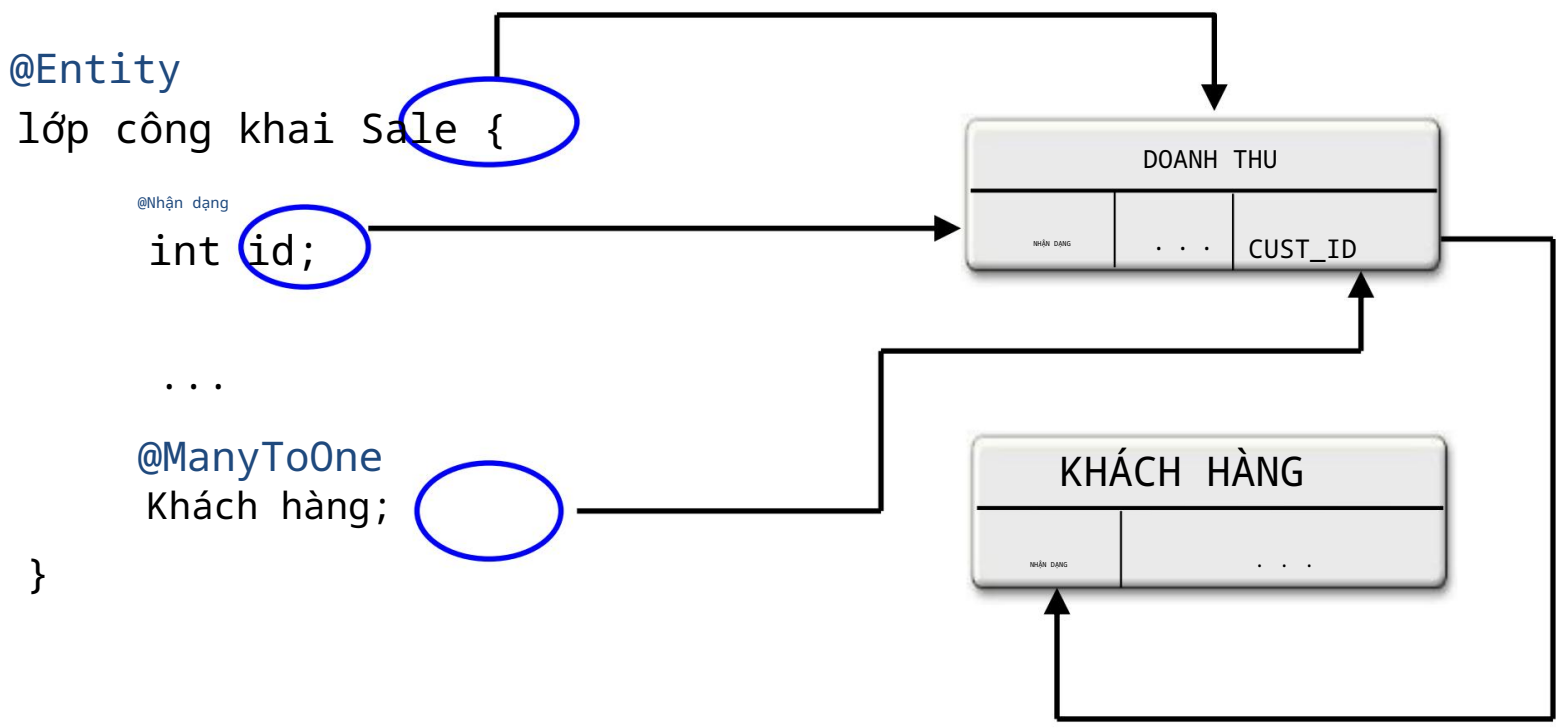
```
@ManyToMany(
```

```
cascade = {CascadeType.PERSIST, CascadeType.MERGE}, fetch = FetchType.EAGER)
```

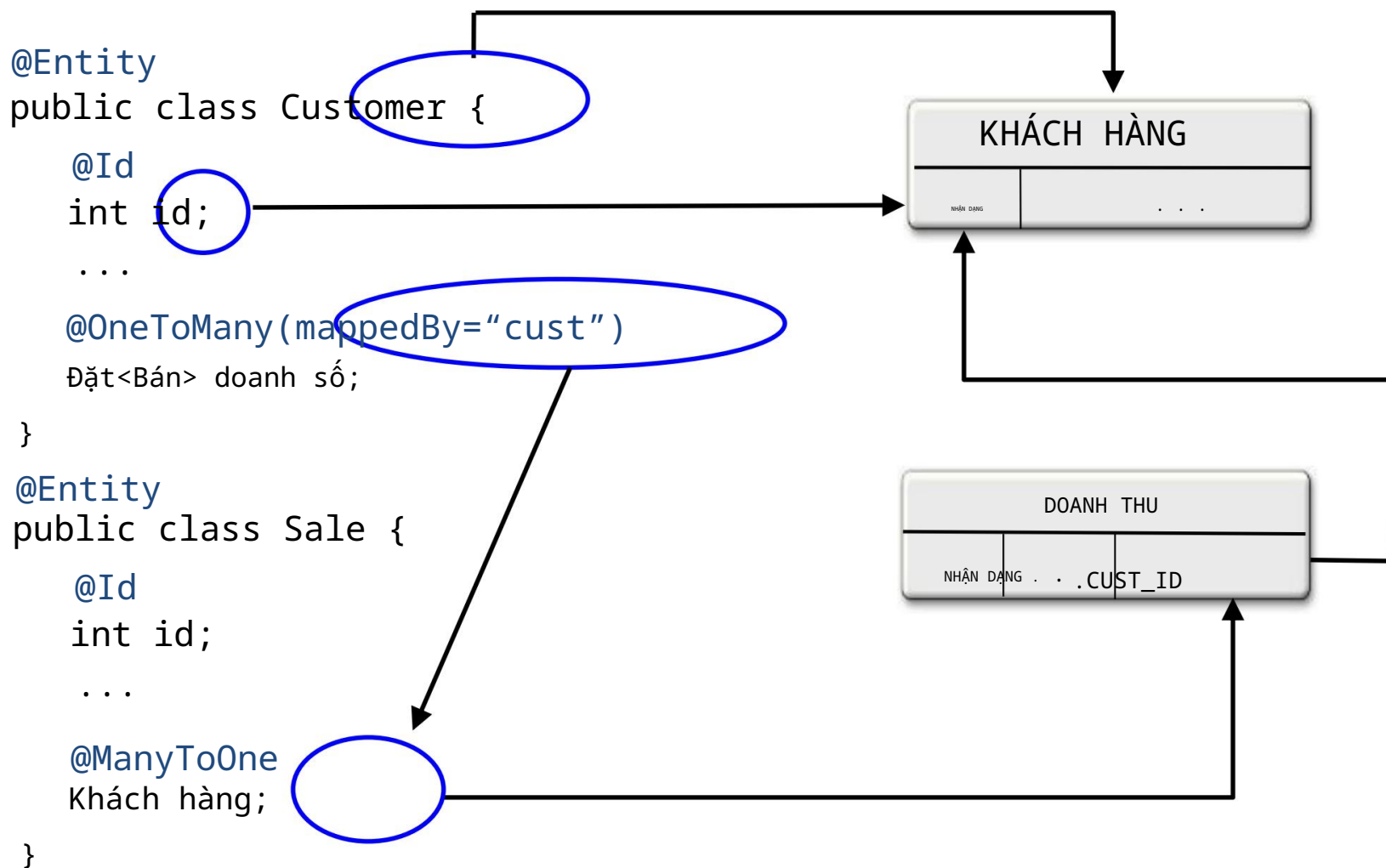
Ảnh xạ đơn giản



Ánh xạ Nhiều-Một



Ảnh xạ Một-Nhiều



Ảnh xạ nhiều-nhiều

```
@Entity
```

```
public class Customer {
```

```
    ...
```

```
@ManyToMany(cascade=CascadeType.ALL)
```

```
@JoinTable(name="CUSTOMER_SALE",
```

```
            joinColumns=@JoinColumn(name="CUSTOMER_ID",
```

```
                                    referencedColumnName="customer_id"),
```

```
            inverseJoinColumns=@JoinColumn(
```

```
name="SALE_ID", referencesColumnName="sale_id")
```

```
    Bộ sưu tập<Bán hàng> bán hàng;
```

```
}
```

```
@Entity
```

```
public class Sale {
```

```
    ...
```

```
@ManyToMany(mappedBy="sales")
```

```
    Bộ sưu tập khách hàng<Customer>;
```

```
}
```



Đơn vị bền vững

Đơn vị lưu trữ

- Một đơn vị lưu trữ (persistence unit) định nghĩa một tập hợp tất cả các lớp thực thể được quản lý bởi các thể hiện EntityManager trong một ứng dụng.
- Mỗi đơn vị duy trì có thể có các nhà cung cấp khác nhau.
và trình điều khiển cơ sở dữ liệu
- Các đơn vị lưu trữ được định nghĩa bởi META-INF

Tập cấu hình /**persistence.xml**

Tệp persistence.xml

- Tệp persistence.xml định nghĩa một hoặc nhiều đơn vị lưu trữ dữ liệu.

```
<persistence-unit name="TemporalConstraint">
  <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
  <class>my.package.MyEntity</class>
  <exclude-unlisted-classes>false</exclude-unlisted-classes>
  <properties>
    <property name="eclipselink.target-database" value="SQLServer" />
    <property name="eclipselink.ddl-generation" value="none" />
    <property name="javax.persistence.jdbc.url"
      value="jdbc:sqlserver://localhost:1433;databaseName=MyDatabase"/>
    <property name="javax.persistence.jdbc.user" value="username"/>
    <property name="javax.persistence.jdbc.password" value="password"/>
    <property name="javax.persistence.jdbc.driver"
      value="com.microsoft.sqlserver.jdbc.SQLServerDriver"/>
  </properties>
</persistence-unit>
```



EntityManager & Ngữ cảnh bền vững

Sử dụng API lưu trữ

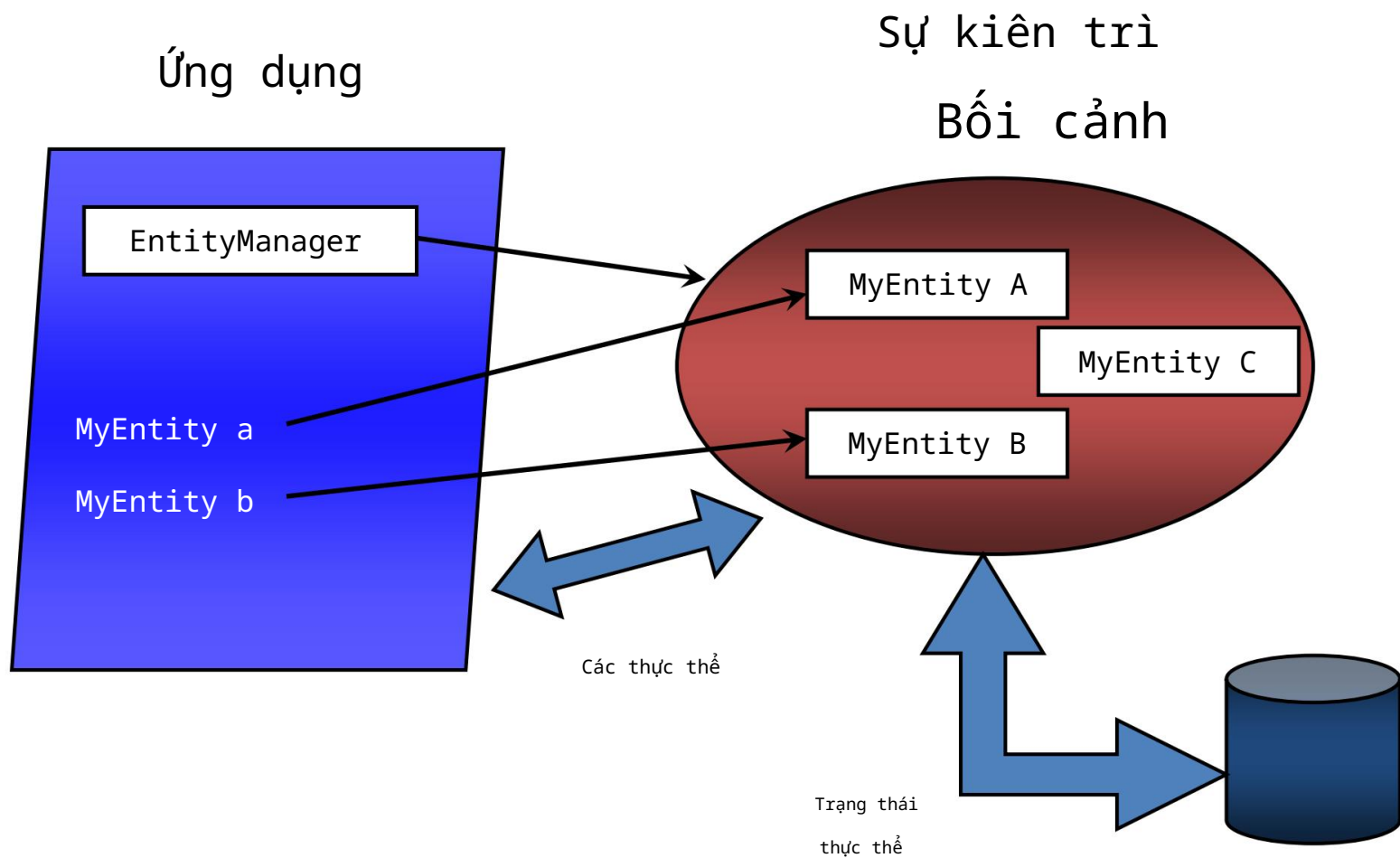
Quản lý các thực thể

- Các thực thể được quản lý bởi **trình quản lý thực thể**
- Trình quản lý thực thể được biểu thị bằng các thể hiện của **javax.persistence.EntityManager**
- Mỗi thể hiện của EntityManager được liên kết với một **ngữ cảnh bền vững**
- Ngữ cảnh lưu trữ xác định phạm vi mà theo đó các thể hiện thực thể cụ thể được tạo ra, lưu trữ, và đã bị xóa bỏ

Bối cảnh bền vững

- Ngữ cảnh bền vững là một tập hợp các thể hiện thực thể được quản lý tồn tại trong một kho dữ liệu cụ thể.
 - o Các thực thể được khóa bằng định danh bền vững của chúng.
 - o Chỉ có thể tồn tại một thực thể với định danh bền vững nhất định trong kho dữ liệu.
- bối cảnh bền vững
 - o Các thực thể được thêm vào ngữ cảnh lưu trữ, nhưng không phải là có thể tháo rời riêng lẻ ("tách rời")
- Được kiểm soát và quản lý bởi **EntityManager**
 - o Nội dung của ngữ cảnh lưu trữ thay đổi do các thao tác trên API EntityManager

Bối cảnh bền vững



Trình quản lý thực thể

- Một thể hiện của `EntityManager` được sử dụng để quản lý

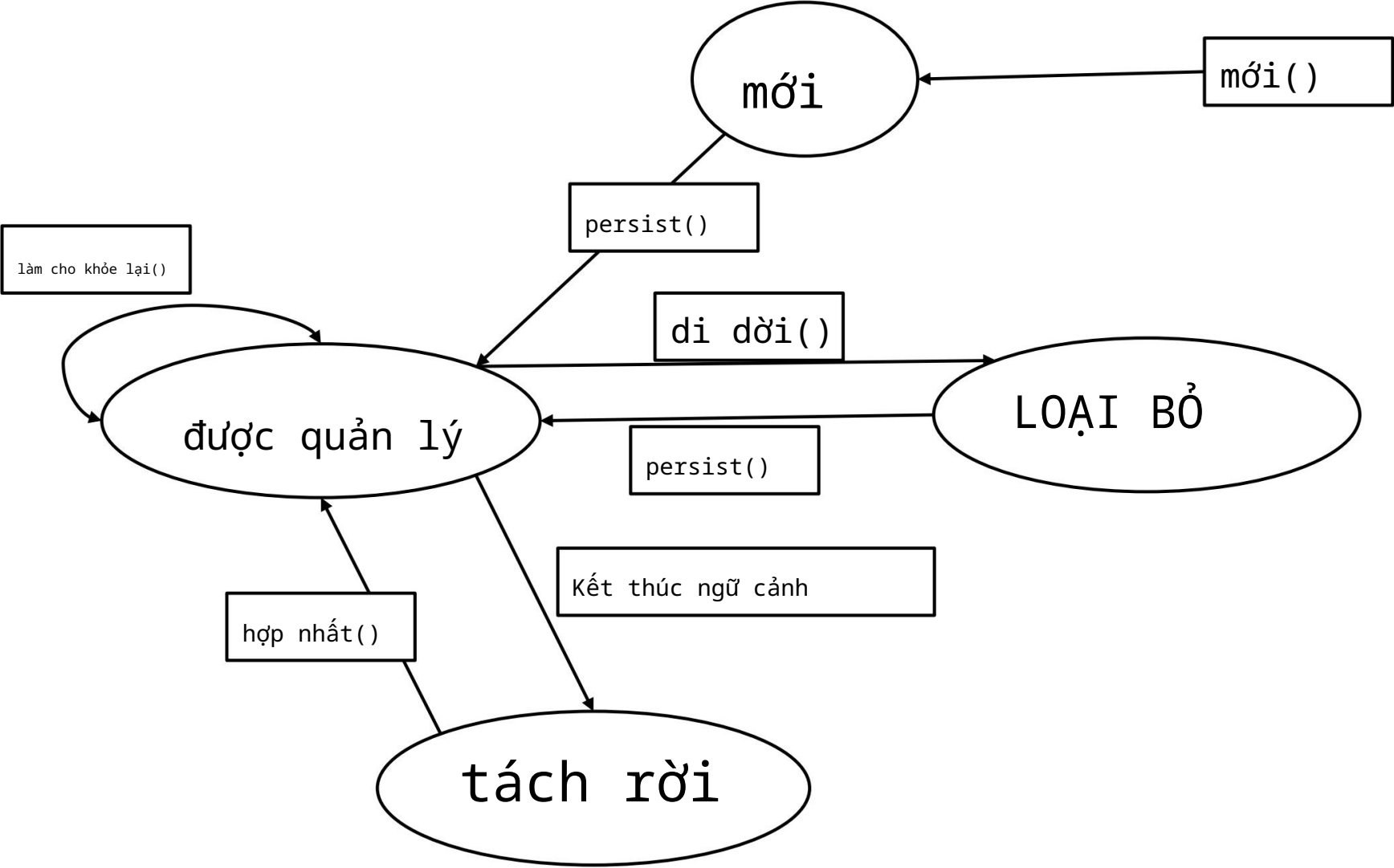
Trạng thái và vòng đời của các thực thể

trong ngữ cảnh lưu

trữ • Các thực thể có thể ở một trong các trạng thái sau:

1. Mới
2. Đã quản lý
3. Tách rời
4. Đã xóa

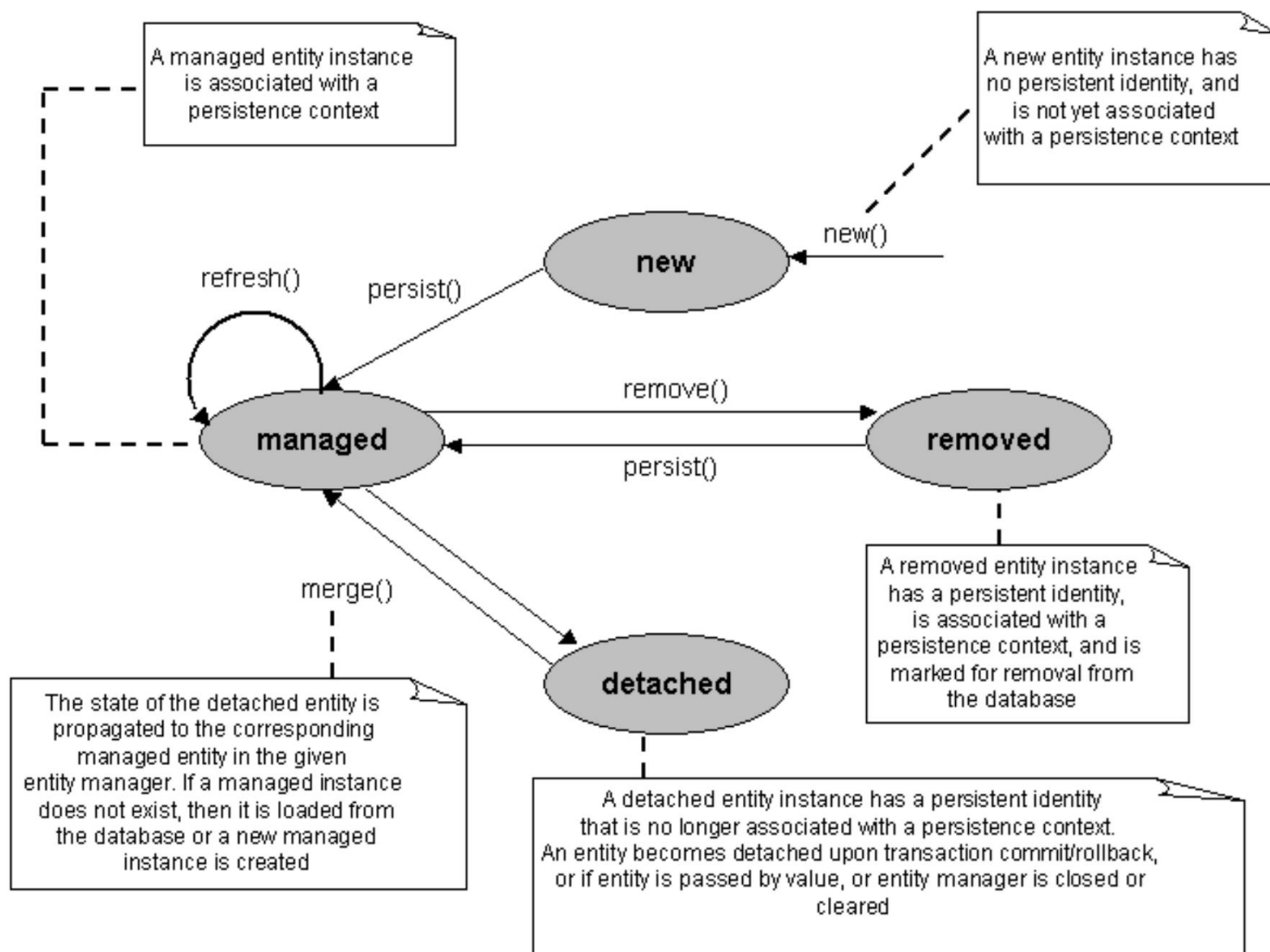
Vòng đời thực thể



Vòng đời thực thể

- **Mới** - thực thể được khởi tạo nhưng chưa được liên kết với cơ chế lưu trữ.
Ngữ cảnh. Không liên kết với cơ sở dữ liệu.
- **Được quản lý** - liên kết với ngữ cảnh lưu trữ. Các thay đổi được đồng bộ hóa với cơ sở dữ liệu.
- **Không kết nối** - có ID nhưng không được kết nối với cơ sở dữ liệu
- **Đã xóa** - liên quan đến ngữ cảnh lưu trữ, nhưng vẫn nằm bên dưới.
Hàng này sẽ bị xóa.
- Trạng thái của các thực thể bền vững được đồng bộ hóa với cơ sở dữ liệu.
khi giao dịch được thực hiện

Vòng đời thực thể



Trình quản lý thực thể

- API của EntityManager: o tạo và xóa các thực thể o tìm kiếm thực thể theo khóa chính của thực thể o cho phép chạy các truy vấn trên các thực thể
- Có hai loại EntityManager: o EntityManager do ứng dụng quản lý • tức là: chạy thông qua Java SE
 - o Trình quản lý thực thể được quản lý bởi container
 - tức là: chạy qua Java EE Container ví dụ: JBossAS, GlassFish, .

Trình quản lý thực thể được quản lý bởi container (JavaEE)

- Với trình quản lý thực thể được quản lý bởi vùng chứa, một
Ngữ cảnh lưu trữ của `EntityManagerInstance` được container tự
động truyền đến tất cả các thành phần ứng dụng sử dụng
`EntityManagerInstance` trong một giao dịch Java Transaction API
(JTA) duy nhất.
- Bộ chứa Java EE quản lý vòng đời của các thành phần được quản lý bởi bộ chứa.
người quản lý thực thể.
- Để có được một thể hiện của `EntityManager`, hãy inject entity manager vào
Thành phần ứng dụng:

```
@PersistenceContext
```

```
private EntityManager em;
```

Trình quản lý thực thể do ứng dụng quản lý (JavaSE)

- Các ứng dụng Java SE tạo các thể hiện EntityManager bằng cách sử dụng **trực tiếp Persistence** và **EntityManagerFactory**:
 - o **javax.persistence.Persistence**
 - Lớp gốc để lấy EntityManager
 - Xác định vị trí dịch vụ nhà cung cấp cho một đơn vị lưu trữ được đặt tên.
 - Gọi phương thức tới nhà cung cấp để lấy EntityManagerFactory
 - o **javax.persistence.EntityManagerFactory**
 - Tạo EntityManager cho một đơn vị lưu trữ hoặc cấu hình được đặt tên.

```
EntityManagerFactory fac =  
    Persistence.createEntityManagerFactory("JPADemo");  
EntityManager em = fac.createEntityManager();
```

Giao dịch thực thể (Trong JavaSE)

- Chỉ được sử dụng bởi các EntityManager cục bộ tài nguyên
- Phân định giao dịch dưới sự kiểm soát ứng dụng rõ ràng bằng cách sử dụng

API Giao dịch Thực thể

`begin(), commit(), rollback(), isActive()`

- Các tài nguyên cơ bản (JDBC) được phân bổ bởi EntityManager như sau:

yêu cầu

```
EntityTransaction trs = entityManager.getTransaction();
try {
    trs.begin();
    //do your works...
    trs.commit();
} catch (Exception e) {
    trs.rollback();
}
```

Các thao tác trên đối tượng thực thể

- Các thao tác API của EntityManager:

- o `persist()` - Lưu thực thể vào cơ sở dữ liệu

- o `remove()` - Xóa thực thể khỏi cơ sở dữ liệu
 - o `refresh()`

- Tải lại trạng thái thực thể từ cơ sở dữ liệu
 - o `merge()` -

- Đồng bộ hóa thực thể đã tách với p/c
 - o `find()` - Tìm kiếm theo khóa

- chính
 - o `createQuery()` - Tạo truy vấn

- sử dụng JP QL động
 - o `createNamedQuery()` - Tạo truy vấn được định

- nghĩa trước
 - o `createNativeQuery()` - Tạo truy vấn SQL "thuần

- túy" gốc. Cũng có thể gọi

- Thủ tục lưu trữ. o

- o `contains()` - Kiểm tra xem thực thể có được quản lý

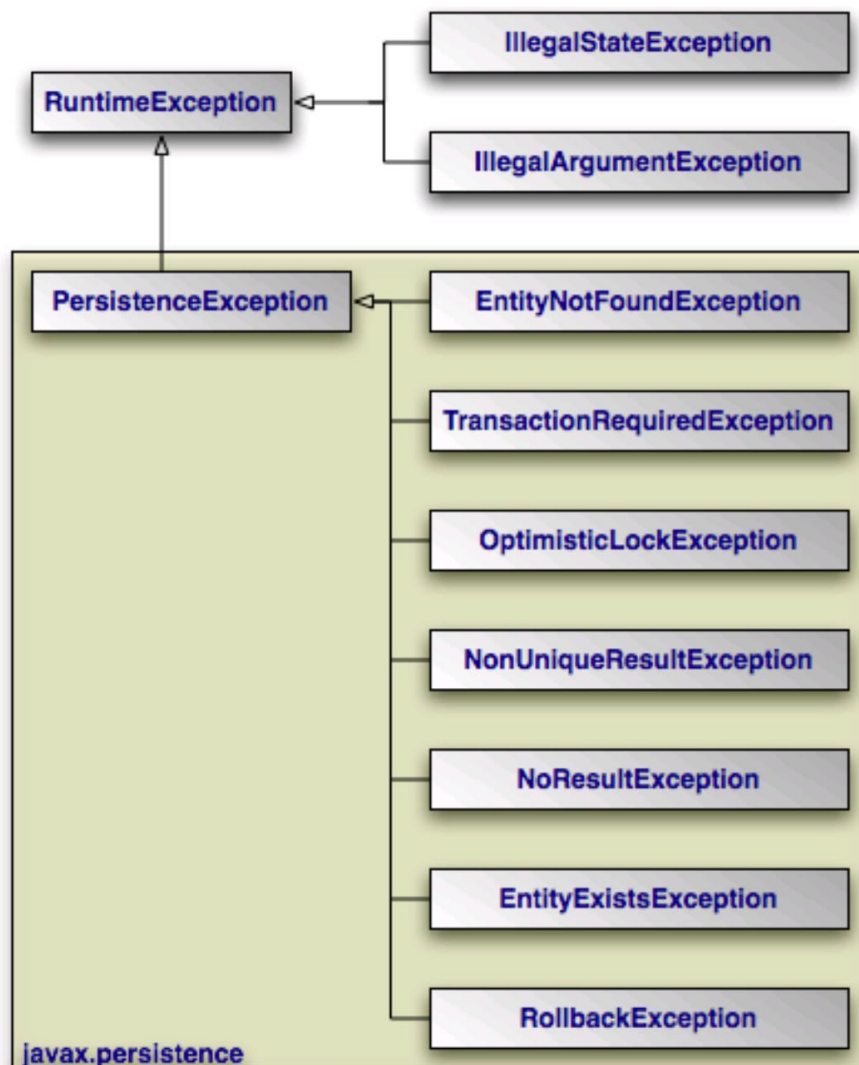
- bởi p/c hay không
 - o `flush()` - Buộc đồng bộ hóa p/c với cơ sở dữ liệu

- Lưu ý: p/c == ngữ cảnh lưu trữ hiện tại



Ngoại lệ JPA

Ngoại lệ JPA



- Tất cả các trường hợp ngoại lệ đều là chưa được kiểm tra
- Các ngoại lệ trong gói `javax.persistence` rất dễ hiểu.



Ngôn ngữ truy vấn JPA (JPAQL)

Ngôn ngữ truy vấn JPA

- JPA có ngôn ngữ truy vấn dựa trên SQL.
- JPQL là một phần mở rộng của EJB QL
- Mạnh mẽ, linh hoạt và hướng đối tượng hơn SQL
- Công cụ lưu trữ dữ liệu phân tích chuỗi truy vấn, chuyển đổi...
Chuyển đổi JPQL sang SQL gốc trước khi thực thi.

Tạo truy vấn

- Các thẻ hiển thị truy vấn được lấy bằng cách sử dụng:

- o `EntityManager.createNamedQuery` (truy vấn tĩnh)

- o `EntityManager.createQuery` (truy vấn động)

- o `EntityManager.createNativeQuery` (truy vấn gốc)

truy vấn:

- o `getResultList()` - thực thi truy vấn trả về nhiều kết quả

- o `getSingleResult()` - thực thi truy vấn trả về một kết quả

- o `executeUpdate()` - thực thi cập nhật hoặc xóa hàng loạt

- o `setFirstResult()` - đặt kết quả đầu tiên cần truy xuất

- o `setMaxResults()` - đặt số lượng kết quả tối đa cần truy xuất

- o `setParameter()` - liên kết một giá trị với tham số được đặt tên hoặc

- tham số vị trí

- o `setHint()` - áp dụng gợi ý dành riêng cho

- nhà cung cấp cho truy vấn

- o `setFlushMode()` - áp dụng chế độ flush cho truy vấn khi nó

Truy vấn tĩnh (có tên)

- Được định nghĩa tĩnh với sự trợ giúp của chú thích `@NamedQuery` cùng với lớp thực thể.
- Các phần tử `@NamedQuery`:
 - o `name` - tên của truy vấn sẽ được sử dụng với Phương thức `createNamedQuery`
 - o `truy vấn` - chuỗi truy vấn

```
@NamedQuery(name="Customer.findAll",  
            truy vấn="SELECT c FROM Customer c")
```

```
Truy vấn findAllQuery = entityManager.createNamedQuery(  
    "findAllCustomers"); List  
customers = findAllQuery.getResultList();
```

Nhiều truy vấn được đặt tên

- Nhiều truy vấn được đặt tên có thể được định nghĩa một cách logic với sự trợ giúp của

Chú thích @NamedQueries

```
@NamedQueries( {  
  
    @NamedQuery(name = "Mobile.selectAllQuery"  
        truy vấn = "SELECT M FROM MOBILEENTITY M"),  
  
    @NamedQuery(name = "Mobile.deleteAllQuery"  
        truy vấn = "XÓA M KHỎI THỰC THỂ DI ĐỘNG M")  
  
}  
)
```

Truy vấn động

- Truy vấn động là các truy vấn được định nghĩa trực tiếp trong logic nghiệp vụ của ứng dụng.
- **Không hiệu quả và chậm hơn.** Công cụ lưu trữ dữ liệu phải phân tích cú pháp và xác thực.

và ánh xạ JPQL sang SQL trong quá trình thực thi

```
public List findAll(String entityName){  
    trả về entityManager.createQuery(  
        "chọn e từ "          + Tên thực thể + " e")  
        .getResultList();  
}
```

Tham số được đặt tên

- Tham số được đặt tên là các tham số trong truy vấn được đặt trước bằng dấu hai chấm (:)
- Để liên kết tham số với một đối số, hãy sử dụng phương thức: o

`Query.setParameter(String name, Object value)`

```
public List findWithName(String name) {  
    trả về em.createQuery(  
        "SELECT c FROM Customer c WHERE c.name LIKE :custName")  
        .setParameter("custName", name)  
        .getResultList();  
}
```


Các tham số vị trí

- Các tham số vị trí được đặt trước bằng **dấu chấm hỏi (?)** và **số thứ tự của tham số trong truy vấn** . Để thiết lập giá trị tham số, hãy sử dụng phương thức:

`Query.setParameter(vị trí số nguyên, giá trị đối tượng)`

```
public List findWithName(String name) {  
    trả về em.createQuery(  
        "SELECT c FROM Customer c WHERE c.name LIKE ?1")  
        .setParameter(1, name)  
        .getResultList();  
}
```

Truy vấn bản địa

- Các truy vấn có thể được thể hiện bằng SQL gốc
- Sử dụng khi bạn cần sử dụng SQL gốc của cơ sở dữ liệu đích
- Có thể gọi các thủ tục lưu trữ bằng cú pháp "call procname"

```
Truy vấn q = em.createNativeQuery(  
    "SELECT o.id, o.quantity, o.item"          +  
    "Từ đơn hàng o, mặt hàng i"          +  
    "WHERE (o.item = i.id) AND (i.name = 'widget')",  
    com.acme.Order.class);
```

Hãy sử dụng chú thích @SqlResultSetMapping cho các trường hợp nâng cao hơn.

Thao tác truy vấn - Nhiều kết quả

- Phương thức `Query.getResultList()` sẽ thực thi một truy vấn và có thể trả về một đối tượng `List` chứa nhiều thể hiện thực thể.

```
Truy vấn query = entityManager.createQuery("SELECT C FROM CUSTOMER");  
  
List<MobileEntity> mobiles = (List<MobileEntity>)query.getResultList();
```

- Sẽ trả về một đối tượng `List` không có tham số.
- Chỉ có thể thực thi trên các câu lệnh `SELECT` chứ không phải `UPDATE` hoặc `XÓA` các câu lệnh
- Đối với câu lệnh khác ngoài câu lệnh `SELECT` trong thời gian chạy
Ngoại lệ `IllegalStateException` sẽ được ném ra.

Thao tác truy vấn - Kết quả đơn

- Một truy vấn trả về một đối tượng thực thể duy nhất

```
Truy vấn singleSelectQuery = entityManager.createQuery(  
    "SELECT C FROM CUSTOMER WHERE C.ID = 'ABC-123'");  
Khách hàng custObj = singleSelectQuery.getSingleResult();
```

- Nếu quá trình khớp không thành công, thì sẽ xảy ra ngoại lệ `EntityNotFoundException`.
được trả lại
- Nếu có nhiều hơn một kết quả khớp trong quá trình thực thi truy vấn, một ngoại lệ
thời gian chạy `NonUniqueResultException` sẽ được ném ra.

Kết quả truy vấn phân trang

```
int maxRecords = 10; int startPosition = 0;
Chuỗi truy vấn = "SELECT M FROM MOBILEENTITY";
trong khi (đúng){
    Truy vấn selectQuery = entityManager.createQuery(queryString);
    selectQuery.setMaxResults(maxRecords);
    selectQuery.setFirstResult(startPosition);
    List<MobileEntity> mobiles =
        entityManager.getResultList(queryString);
    if (mobiles.isEmpty()){ break; }
    process(mobiles);                // xử lý các thực thể di động
    entityManager.clear();           // tách các đối tượng di động
    startPosition = startPosition + mobiles.size();
}
```

Làm mới các đối tượng truy vấn

- Hai chế độ làm mới các đối tượng truy vấn

- **AUTO** (mặc định) và **COMMIT**

- **TỰ ĐỘNG** - mọi thay đổi được thực hiện đối với các đối tượng thực thể sẽ được phản ánh.

lần tiếp theo khi thực hiện truy vấn **SELECT**

- **COMMIT** - công cụ lưu trữ dữ liệu chỉ có thể cập nhật toàn bộ trạng thái.

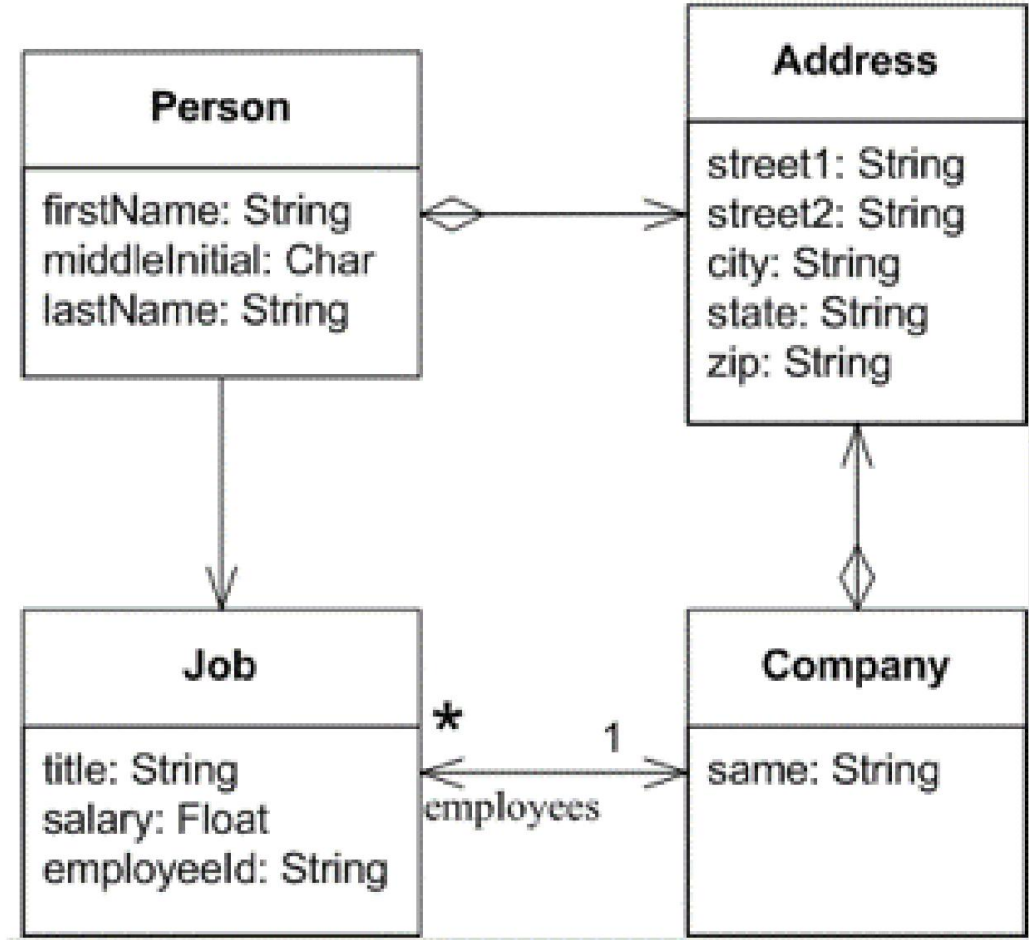
của các thực thể trong quá trình **COMMIT** cơ sở dữ liệu

- được thiết lập thông qua `Query.setFlushMode()`

Bài tập

1. Định nghĩa một lớp thực thể Sinh viên có các thuộc tính Id và FirstName.
và Họ.
2. Định nghĩa một lớp thực thể có ID, tên và danh sách sinh viên.
3. Tạo cơ sở dữ liệu phù hợp với các lớp thực thể. Sử dụng Apache Derby và tính năng hỗ trợ cột định danh tích hợp sẵn của nó.
4. Tạo một chương trình liệt kê tất cả các lớp học và học sinh trong các lớp đó.
mỗi lớp.
5. Tạo một chương trình thêm một lớp học mới và một vài học sinh.
Bên trong nó.

Bài tập



Bản tóm tắt

API lưu trữ dữ liệu Java

- o Các thực thể
- o EntityManager & Ngữ cảnh bền vững
- o Đơn vị lưu trữ
- o Ngoại lệ o

Ngôn ngữ truy vấn JPA

Buổi học này đến đây là kết thúc!

Cảm ơn tất cả mọi người đã quan tâm và kiên nhẫn!